System Design Interview: Design ChatGPT

#125: System Design Interview

NEO KIM AND HAYK

FEB 25, 2026 PAID

Get my system design playbook for FREE on newsletter signup:

Subscribed

- *Share this post & I'll send you some rewards for the referrals.*
- *Block diagrams created using Eraser.*

Most engineers think designing an AI chat system is just *"calling OpenAI API and saving messages to a database".*

That approach doesn't work when you need to handle ChatGPT's scale:

- *5.8 billion visits per month*
- *1 billion weekly active users sending 2.5+ billion prompts per day*

Your database can't store 500TB per year. API costs will hit $145 million per month. And you'll need 11.4 million concurrent connections that no single server can handle.

This problem shows up in senior and staff-level interviews at companies building AI products because it tests the exact skills that separate mid-level engineers from seniors:

- handling stateful connections at scale,
- managing expensive external dependencies,
- keeping costs under control while maintaining good UX,
- and designing for failures when parts of your system go down.

We're going to build this from scratch at ChatGPT's real scale, starting with clarifying requirements, then moving through frontend and backend design, database sharding[1], the complete user flow, GPU infrastructure, and finally explaining how to scale even further to 1 billion users.

Before we design anything, we need to understand exactly what we're building and at what scale...

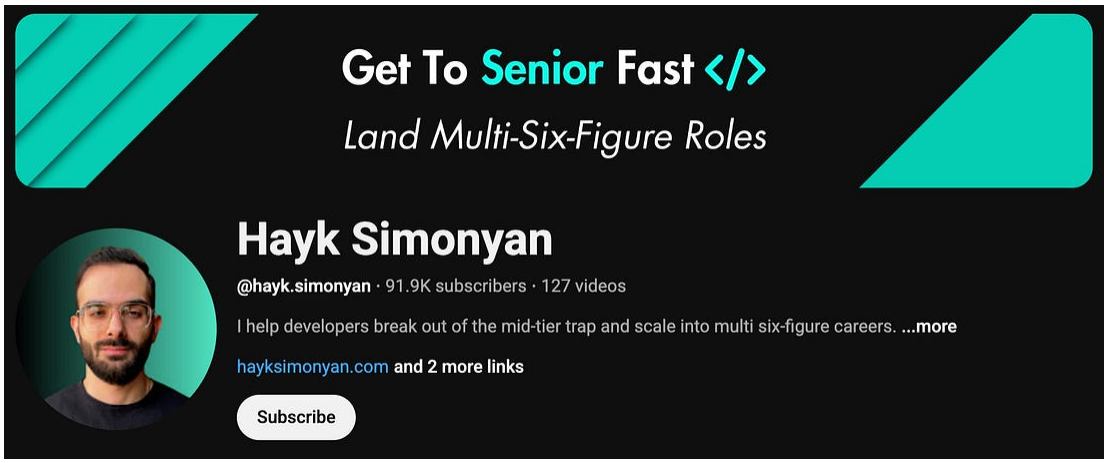
Unblocked: The context layer your AI tools are missing (Partner)

Give your agents the understanding they need to generate reliable code, reviews, and answers. **Unblocked** builds context from your team's code, PR history, conversations, documentation, planning tools, and runtime signals. It surfaces the insights that matter so AI outputs reflect how your system actually works.

Try Unblocked Now

(Thank you, Unblocked, for partnering on this post.)

I want to reintroduce Hayk Simonyan as a guest author.



He's a senior software engineer specializing in helping developers break through their career plateaus and secure senior roles.

If you want to master the essential system design skills and land senior developer roles, I highly recommend checking out Hayk's **YouTube channel**.

His approach focuses on what top employers actually care about: system design expertise, advanced project experience, and elite-level interview performance.

Clarifying Requirements

In interviews, candidates who skip this step fail immediately.

Questions to ask:

- Are we building the AI model itself or integrating with existing ones?
- Do we need streaming responses (word by word) or complete responses?
- What's our expected scale? Thousands or millions of users?
- Do users need accounts and conversation history?
- Are conversations private or shareable?
- Do we support multiple languages?
- How do we handle rate limiting[2]? Free vs paid tiers?
- Text only, or images and files too?
- Do we need content moderation?

For this design, let's assume:

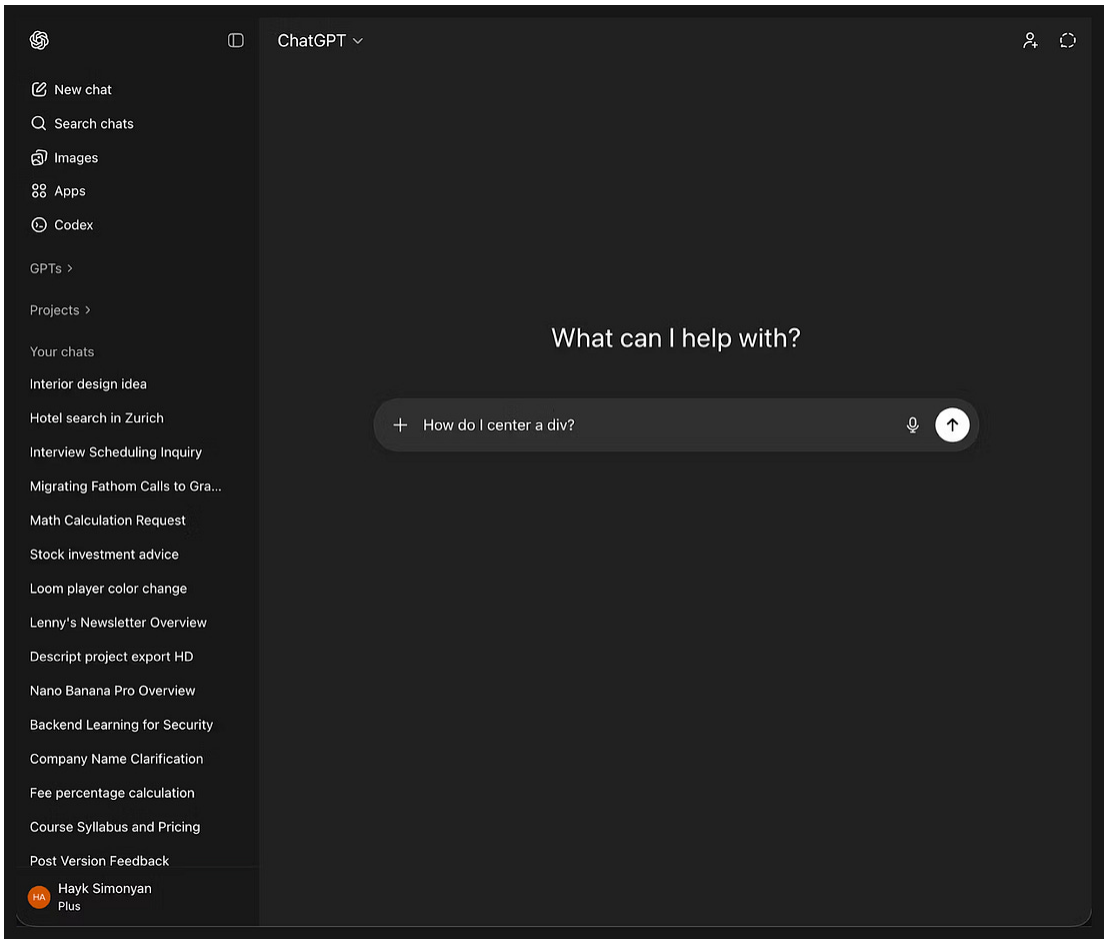
- Integrating with existing LLM (not building the model ourselves)
- Streaming responses required for good UX
- As of January 2026, ChatGPT has 800-900 million weekly active users, with an average of 2.5+ billion prompts per day
- Users need accounts, private conversations with history
- Text only, English first
- Rate limiting on both requests and tokens
- Basic content moderation required

Now that we've defined our requirements, let's start with what users actually see and interact with.

Frontend Interface Design

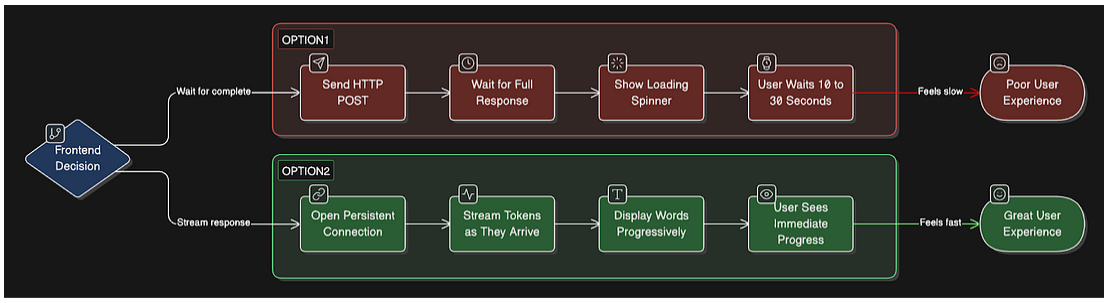
Let's visualize what we're building. This clarifies requirements and influences backend decisions.

Main Interface:



Critical frontend decision: Streaming

You have two options:



Option 1: Wait for complete response

- Simple HTTP POST, and get a full response back
- User waits 10-30 seconds staring at the loading spinner
- Poor UX, feels slow

Option 2: Stream word by word

- Requires a persistent connection
- User sees progress immediately
- Much better UX

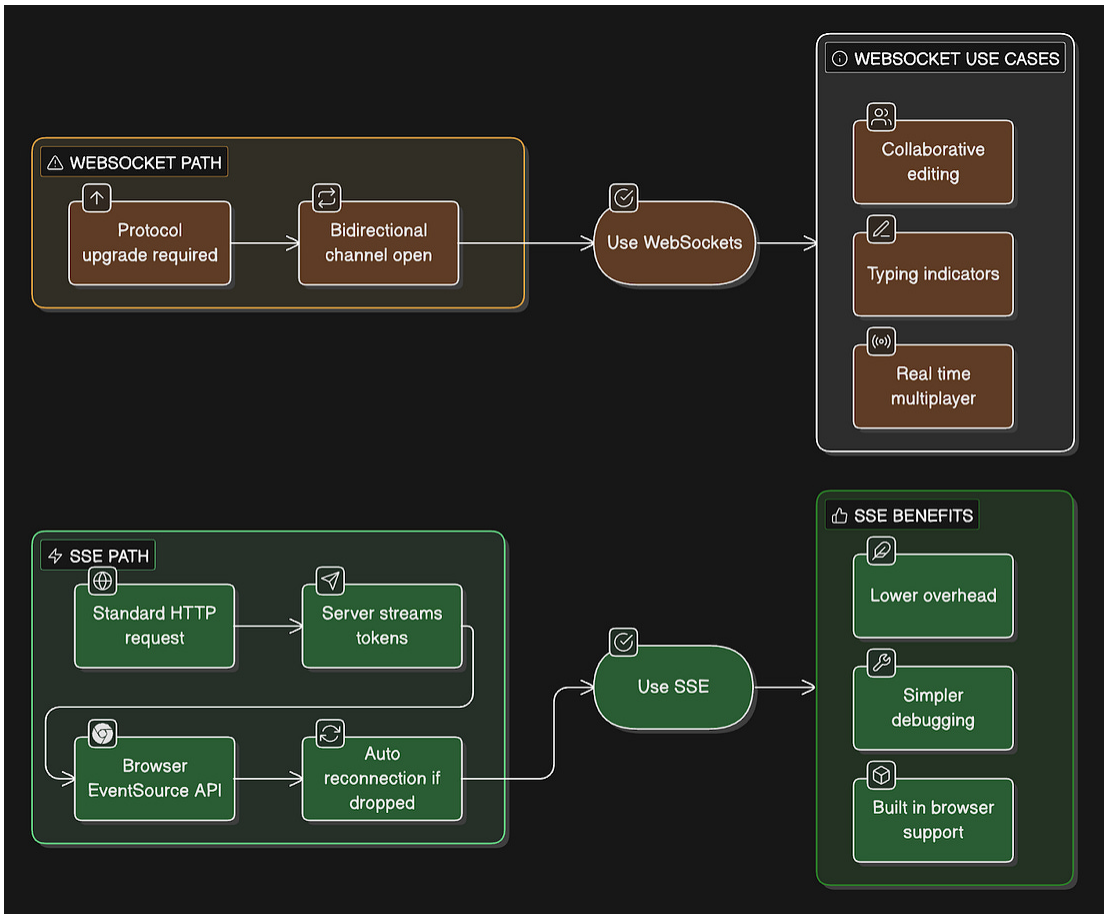
ChatGPT, Claude, and Gemini all use streaming.

Let me verify the exact implementation.

How streaming actually works:

All major AI chat products use Server-Sent Events[3] (SSE), not WebSockets.

Let me explain why:



SSE (what ChatGPT uses):

- Works over standard HTTP
- Browser has a built-in EventSource API
- Auto-reconnection handled automatically
- One-way communication (server to client)
- Simpler to implement and debug
- Lower overhead than WebSockets

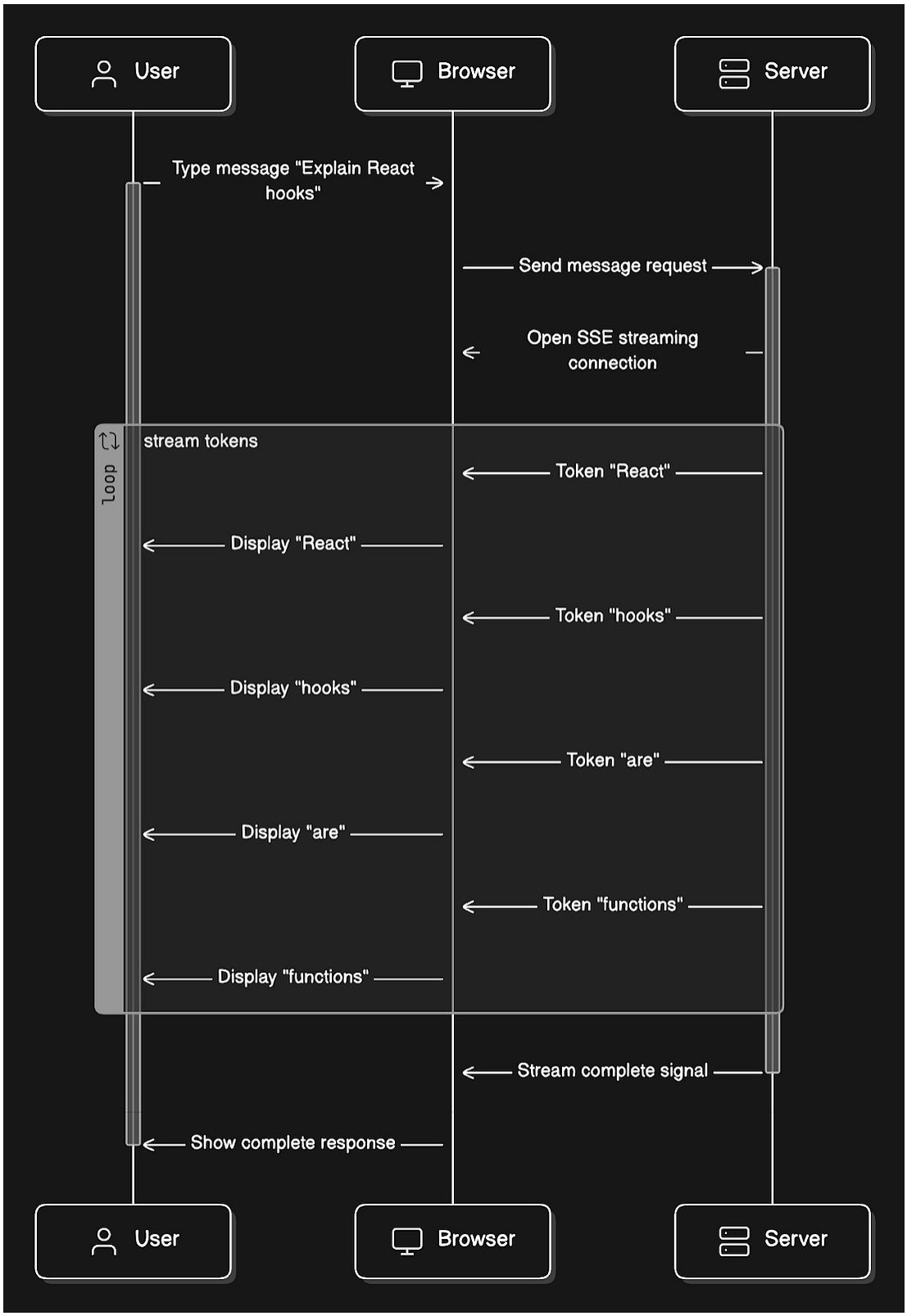
WebSockets (what ChatGPT doesn’t use):

- Requires protocol upgrade from HTTP
- Bidirectional communication

- More complex to implement
- Useful for collaborative features, typing indicators
- Overkill for just streaming AI responses

Example of SSE streaming:

User types: *"Explain React hooks".*



Each word appears in the UI as it arrives, creating the typing effect.

Frontend tech stack:

- React for UI components
- EventSource API for SSE connection
- State management (Zustand or Redux) for conversations
- Optimistic updates (show user message immediately)
- Error handling for network failures, rate limits

The frontend sets user expectations for real-time streaming.

Now, let's formalize what our system must do.

Functional Requirements

- User registration and authentication
- Create and manage conversations
- Send messages and receive streaming AI responses via SSE
- Save all conversations with full history
- Retrieve past conversations
- Stream responses token[4] by token
- Rate limiting (requests + tokens)
- Content moderation for harmful requests

We've defined what the system does. But how well must it perform?

These requirements will drive our architectural decisions.

Non-Functional Requirements

How well it must perform (we'll connect these to design decisions later):

- Availability: 99.9%
    - Max 43 minutes of downtime per month
- Latency: First token < 2 seconds
    - Users wait no more than 2 seconds
- Scalability: 200M DAU, 20M concurrent conversations
    - Assuming 10% concurrency
- Consistency: Zero message loss
    - Every message must be saved
- Cost efficiency
    - LLM APIs are expensive and need optimization
- Security: End-to-end encryption
    - Private conversations

We'll show how our design achieves each of these.

We've set our performance targets. Now let's run the numbers to see what infrastructure we actually need.

Capacity and Storage Estimation

Math matters because it drives infrastructure decisions.

Let's use ChatGPT's actual scale of growth projections:

Traffic:

- 1 billion weekly active users (WAU)
    - The current sources show 800-900 million weekly active users.
    - To accommodate growth and simplify our calculations, let's design for 1 billion weekly active users.

- ~200 million daily active users (DAU)
- 2.5 billion prompts/messages per day
    - ChatGPT's actual processing volume
- 12.5 messages per user per day average
- Peak hours (20% of daily traffic in 2 hours)
    - 69,444 messages/second
- Concurrent conversations at peak: ~20 million
    - Assuming 10% of DAU are chatting simultaneously

Storage per conversation:

- User message: 100 characters = 100 bytes
- AI response: 500 characters = 500 bytes
- Per exchange: 600 bytes
- 12.5 exchanges/day/user: 7.5 KB/user/day
- 200M DAU × 7.5KB = 1.5TB/day
- Annual: 548 TB/year

Metadata storage:

- User profiles (1KB each): 1B × 1KB = 1TB
- Conversation metadata (500 bytes each, 10 per user average): 10B conversations × 500B = 5TB

Total Year 1: ~554TB (need distributed database, single PostgreSQL won't cut it)

Connection overhead:

- 20M concurrent SSE connections at peak
- Each connection: ~10-30 seconds average

- Need infrastructure that can handle millions of long-lived connections

Bandwidth:

- Average response: 750 bytes streamed over 20 seconds
- 20M concurrent: 20M × 750B / 20s = 750 MB/s = 6 Gbps
- Peak outbound bandwidth requirement

Key insight: At ChatGPT's scale, single-server solutions don't work. We need distributed systems, sharding, and multi-region deployment from day one.

Now that we know the scale we're dealing with, let's design the actual system that can handle it.

High-Level System Design

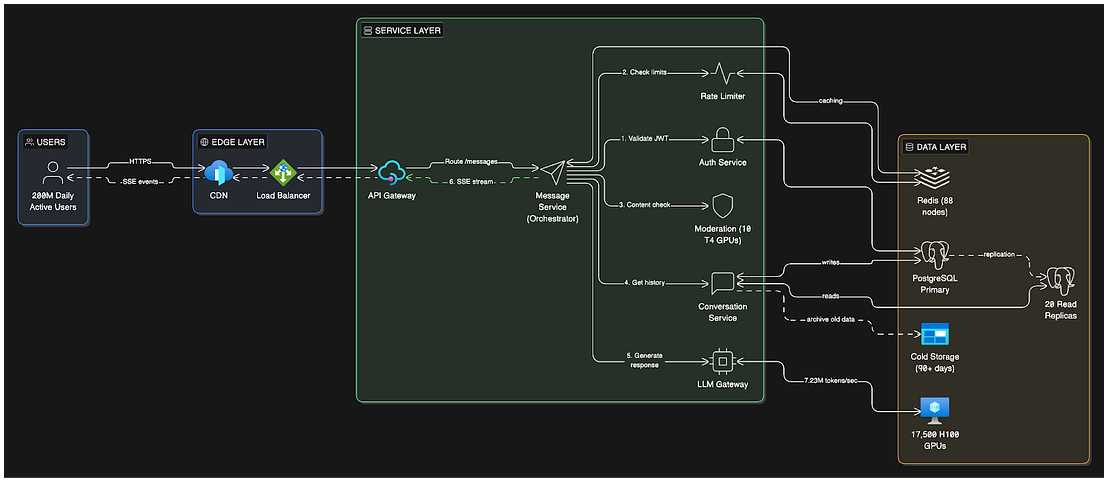
Before diving into costs and implementation details, let's map out the complete system architecture.

OpenAI uses a multi-cloud setup to secure enough compute power and avoid relying on a single vendor:

- **Microsoft Azure:** Still their leading provider. They have a massive, long-term partnership in which Microsoft builds specialized supercomputers for training OpenAI's models.
- **AWS:** OpenAI signed a $38 billion deal with Amazon in late 2025. They now run significant workloads on AWS's GPU clusters and specialized AI hardware.
- **Google Cloud (GCP):** As of mid-2025, OpenAI uses GCP to power parts of ChatGPT Enterprise and the Team tiers, as well as their API.

- **Oracle:** They have a multi-billion dollar agreement to use Oracle's data center capacity, specifically for the massive "Stargate" infrastructure project.
- **CoreWeave:** They also use this specialized "neocloud" provider for dedicated access to high-performance NVIDIA GPUs.

Components Overview



Our system has three main layers:

1. Client Layer
    - React web application
    - Mobile apps (iOS/Android)
    - SSE connections for real-time streaming
2. Service Layer (Microservices[5])
    - API Gateway: Entry point, SSL termination, request routing
    - Auth Service: JWT[6] authentication, user session management
    - Message Service: Core orchestrator for message processing and streaming

    - Conversation Service: CRUD operations for conversations and message history
    - Rate Limiter Service: Track and enforce request/token limits per user
    - Moderation Service: Content filtering for harmful requests
    - LLM Gateway Service: Routes requests to GPU clusters, handles streaming responses

- *Why this order?*
    - Load Balancer is "dumb" - it just distributes TCP/HTTP connections across servers.
    - API Gateway is "smart" - reads the URL path and routes: `/auth/*` → Auth Service, `/conversations/*` → Conversation Service, etc.

3. Data Layer
    - PostgreSQL: Single primary + 20 read replicas[7] (user data, conversations, messages)
    - Redis[8] Cluster: Rate limiting state, conversation history cache, response cache
    - GPU Cluster: 17,500 H100[9] GPUs for model inference
    - Azure Blob Storage: Long-term archive for old conversations

How Does the User Receive the Response?

The response flows back through the SAME connection path, reversed:

- The HTTP connection stays open from:

```
Browser → Load Balancer → API Gateway → Message Service
```

- Message Service writes tokens to the response stream
- Each token flows backward:

```
Message Service → API Gateway → Load Balancer → Browser
```

- Browser's EventSource API receives each event in real-time
- Connection closes when streaming completes

Think of it like a phone call:

```
You call (request) → they answer and KEEP TALKING for 20 seconds (streaming)
→ then hang up.
```

Same connection, just stays open longer.

Key Architectural Decisions

Single Primary PostgreSQL (not sharded)

- Why: ChatGPT's workload is 90% reads, 10% writes
- OpenAI's proven architecture: handles 800M users without sharding
- Writes: All go through a single primary instance
- Reads: Distributed across 20+ replicas in 6 regions
- Trade-off: Single writer bottleneck, but application simplicity wins

Why reads dominate despite 2.5B prompts/day

Think about what happens per user prompt:

- Load user account
- Load subscription/quota info
- Load conversation metadata
- Load recent messages for context
- Check rate limits

- Maybe load feature flags, experiments, and so on.

Those are all *reads*.

Writes usually happens only when:

- A new message is stored
- Conversation metadata is updated
- Usage counters are incremented
- Logs are persisted

So one prompt can trigger many reads, but only a few writes.

Microservices Architecture

- Why: Independent scaling and deployment
- Each service scales independently based on its load
- LLM Gateway can scale GPU clusters without touching other services
- Rate Limiter needs different scaling than Conversation Service
- Trade-off: Operational complexity vs scaling flexibility

SSE for Streaming (not WebSockets)

- Why: Simpler, one-way communication is sufficient
- Browser has a built-in EventSource API with auto-reconnection
- Lower overhead than WebSockets
- Most AI products (ChatGPT, Claude, Gemini) use SSE
- Trade-off: Can’t do bidirectional communication (acceptable for chat)

Self-Hosted GPU Infrastructure

- Why: At 2.5B messages/day, API costs would be $159M/month
- Self-hosting: $52.4M/month (67% cheaper)
- Requires: 17,500 H100 GPUs across multiple cloud providers
- Trade-off: $525M upfront cost vs ongoing API expenses

Multi-Region Deployment

- 6 regions: 2 Americas, 2 Europe, 2 Asia-Pacific
- Why: Global low-latency (users connect to the nearest region)
- Each region has a full application stack + read replicas
- GPU clusters are centralized in 3-4 major data centers
- Trade-off: Operational complexity vs user experience

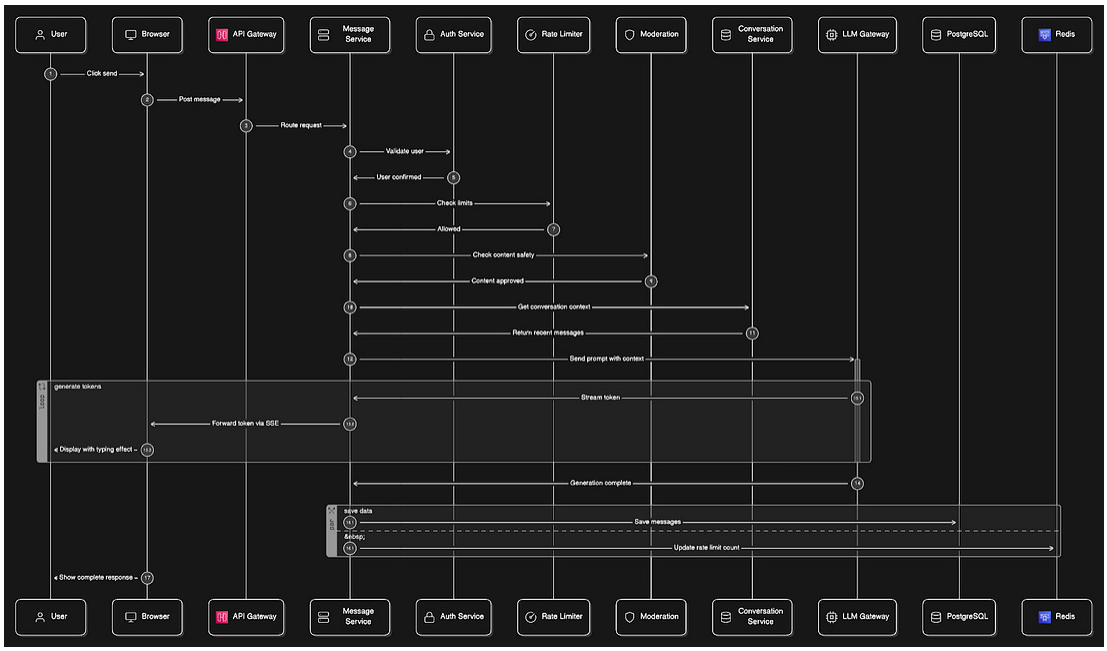
We've mapped out the architecture.

The interesting part comes next: how much this actually costs to run, why OpenAI made the controversial choice to not shard their database, and what changes at 500M and 1B users.

First, let's trace a message from the send button to the AI response...

Request Flow: User Sends Message

Step-by-step flow when user sends *"Explain React hooks"*:



1. User clicks Send
    - Browser: POST /api/conversations/conv_123/messages
    - Payload: {"message": "Explain React hooks"}
    - Hits the nearest Azure Front Door edge location
2. API Gateway receives the request
    - SSL termination[10]
    - Extract JWT from the Authorization header
    - Route to Message Service in the nearest region
3. Message Service coordinates (the orchestrator)
    - Calls Auth Service: Validate JWT → returns user_id
    - Calls Rate Limiter: Check if user is under limits → allow/deny
    - Calls Moderation Service: Check content safety → pass/block
    - Calls Conversation Service: Fetch last 10 messages for context
    - Calls LLM Gateway: Send prompt + context for inference
4. LLM Gateway streams response

    - Routes requests to the available GPU in the cluster
    - Model generates tokens: “React”, “hooks”, “are”, “functions”...
    - Streams each token back to the Message Service
5. Message Service streams to the user
    - Opens an SSE connection to the browser
    - Forwards each token: event: token, data: “React”
    - User sees typing effect in real-time
    - When complete: event: done, data: {”messageId”: “msg_456”}
6. Message Service saves to the database
    - Writes the user message to PostgreSQL primary
    - Writes the assistant response to PostgreSQL primary
    - Updates conversation updated_at timestamp
    - Updates the rate limiter token count in Redis

Total latency breakdown:

- Auth check: 20ms
- Rate limit check: 5ms
- Moderation: 100ms
- Fetch conversation history: 50ms
- First token from LLM: 1.5 seconds
- Total to first token: ~1.7 seconds (under 2-second target)

How Does This Architecture Scale?

- Read scaling:
    - Add more PostgreSQL replicas as read traffic grows.
    - OpenAI uses ~50 replicas.
- Write scaling:

    - Single primary handles 28,935 writes/second.
    - If this becomes a bottleneck, offload write-heavy workloads to separate systems (like OpenAI does with Cosmos DB).
- Compute scaling:
    - Add more application servers to handle SSE connections.
    - Currently, 8,000 servers for 20M concurrent.
- GPU scaling:
    - Add more H100s to the cluster as inference demand grows.
    - Currently 17,500 GPUs for 7.23M tokens/second.
- Geographic scaling:
    - Add new regions to reduce latency for users in new markets.

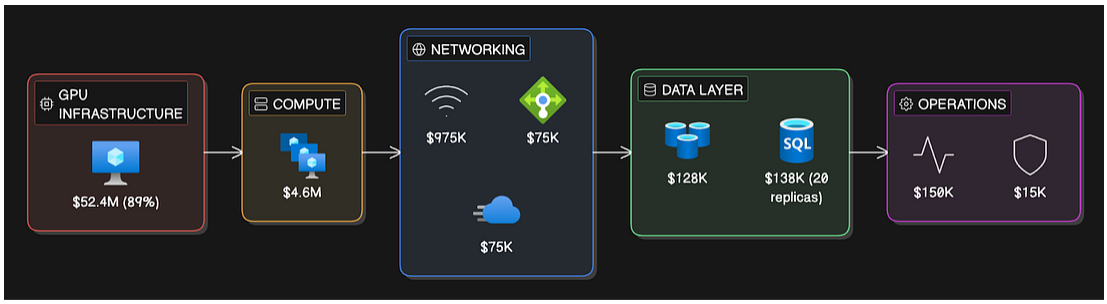
The architecture is proven at scale by OpenAI and can grow from 200M DAU to 500M+ DAU without a fundamental redesign.

We've mapped out the architecture.

Now let's break down what it actually costs to run ChatGPT at this scale.

Cost Estimation

Let's calculate every cost, not just the LLM API:



1. LLM API Costs (if using API):

If we were building a wrapper around OpenAI's API, like many startups do:

GPT-4o pricing: $2.50 per 1M input tokens, $10.00 per 1M output tokens

- 2.5 billion messages/day
- Input: 50 tokens average (message + context)
- Output: 200 tokens average
- Daily input: 125B tokens = $312,500
- Daily output: 500B tokens = $5,000,000
- Daily cost: $5,312,500
- Monthly: $159.4 million

This is absolutely massive. OpenAI would go bankrupt if it used its own API.

Optimization strategies:

- Use GPT-4o-mini for simple queries (80% cheaper: $0.15/$0.60 per 1M)
- Aggressive caching for common questions
- Classify query complexity, route to the appropriate model
- Assuming 60% can use the mini model:
    - 40% GPT-4o: $63.8M/month

- 60% GPT-4o-mini: $4.8M/month
- Optimized: ~$68.6M/month

But this is still unsustainable. How does ChatGPT actually do it?

OpenAI doesn't call its own API. They run direct GPU inference[11]:

- Deploy models on their own GPU clusters (NVIDIA A100s, H100s)
- Direct model serving, no API overhead
- Cost per token is compute time, not API pricing
- Economies of scale: one H100 GPU can serve many users simultaneously
- Hardware cost amortized across millions of requests

For our design, we're building ChatGPT from scratch with direct GPU inference (not using the API). At this scale, API usage is economically impossible.

2. GPU Infrastructure (for self-hosted model inference):

At 200M DAU with 2.5B messages/day:

GPU calculation:

- Daily messages: 2.5 billion
- Average tokens per request: 250 tokens total (50 input + 200 output)
- Daily tokens: 2.5B × 250 = 625 billion tokens/day
- Per second: 625B ÷ 86,400 seconds = 7.23 million tokens/second
- H100 GPU throughput: ~1,000 tokens/second per GPU (for GPT-4 class models)
- Peak load (3x average for spikes): 7.23M × 3 = 21.7M tokens/second

- GPUs needed: 21.7M ÷ 1,000 = 21,700 GPUs at peak
- Target 80% utilization: 21,700 ÷ 0.8 = 27,125 GPUs

Yet with smart batching and caching:

- 30% of requests cached (skip GPU): 27,125 × 0.7 = 18,987 GPUs
- Round to: ~17,500 H100 GPUs (conservative estimate)

At 200M DAU with 2.5B messages/day:

- Need: ~17,500 H100 GPUs to handle peak load
- Hardware cost: 17,500 × $30k = $525M upfront (amortized over 3 years = $14.6M/month)
- Operating cost: 17,500 GPUs × $3/hour × 24h × 30 days = $37.8M/month
    - The $3/hour includes GPU compute, power, and cooling infrastructure
- Total GPU infrastructure: ~$52.4M/month

This is 24% cheaper than using the optimized API ($68.6M vs $52.4M) and 67% cheaper than using the unoptimized API ($159.4M).

3. Database (PostgreSQL single primary + read replicas):

Read replica calculation:

- Daily read queries: Assume 10 reads per message (fetch history, load conversations)
- Total reads: 2.5B messages × 10 = 25 billion reads/day
- Per second: 25B ÷ 86,400 = 289,351 reads/second
- Peak (3x): 868,053 reads/second
- Each db.r6g.8xlarge[12] replica handles: ~45,000 reads/second sustainable

- Replicas needed: 868,053 ÷ 45,000 = ~19.3 replicas
- Round to: 20 replicas minimum

Note: OpenAI uses ~50 replicas for 800M weekly users. At our 200M DAU scale, 20 replicas are conservative.

At 554 TB/year storage:

- 1 primary instance: db.r6g.16xlarge (64 vCPU, 512GB RAM): $5,840/month
- Storage: 600TB × $0.115/GB: $69,000/month
- Read replicas (20 replicas across 6 regions):
  - Each: db.r6g.8xlarge (32 vCPU, 256GB RAM): $2,920/month
  - Total for replicas: 20 × $2,920 = $58,400/month
- Backup storage: $5,000/month
- Total database: $138,240/month

4. Application Servers:

OpenAI uses a multi-cloud strategy but primarily runs on Microsoft Azure (their main investor and partner).

For this design, we'll use Azure pricing for most services. However, OpenAI also uses AWS ($38B deal in 2025), Google Cloud (for ChatGPT Enterprise), Oracle (for Stargate project), and CoreWeave (for specialized GPU access) to avoid vendor lock-in and secure enough compute capacity.

Connection capacity calculation:

- Peak concurrent SSE connections: 20M (10% of 200M DAU)
- Each server handles: ~2,500 concurrent SSE connections
  - Limited by: memory (each connection ~25KB), CPU for streaming, network I/O

    - c6i.4xlarge[13]: 16 vCPU, 64GB RAM = 2,500 connections is the safe limit
- Servers needed: 20M ÷ 2,500 = 8,000 servers
- Distributed across 6 regions: ~1,333 servers per region

Azure Standard_D16s_v5[14] (16 vCPU, 64GB RAM): $576/month each

Total: 8,000 × $576 = $4,608,000/month.

5. Load Balancers:

Multi-region deployment for global coverage:

- Americas: 2 regions (US East, US West)
    - Covers North & South America
- Europe: 2 regions (West Europe, North Europe)
    - Covers EU & Middle East
- Asia-Pacific: 2 regions (Southeast Asia, East Asia)
    - Covers Asia & Australia

Total: 6 regions for global low-latency coverage

- Azure Application Gateway + Azure Front Door across regions
- Azure Front Door Premium (global): $35,000/month
- Application Gateway per region: 6 × $5,000 = $30,000/month
    - Each region needs a separate Gateway for local traffic routing
- Request processing at scale: $10,000/month

Total: $75,000/month

6. Redis (rate limiting + caching):

Redis Cluster sizing:

- Rate limiting: Need to handle 200M users checking limits
- Each Redis node: ~5,000 operations/second sustainable
- Peak load: 69,444 requests/second (from traffic calculation)
- Safety factor: 3x for bursts = 208,332 ops/second needed
- Nodes required: 208,332 ÷ 5,000 = ~42 nodes minimum
- Add redundancy (primary + replica): 42 × 2 = 84 nodes
- Round up for clean sharding: 88 nodes

Alternative calculation:

- Split by user ID hash (consistent hashing[15])
- 200M DAU ÷ 88 nodes = ~2.27M users per node
- Each node tracks rate limits for its subset of users

Azure Cache for Redis Enterprise: 88 nodes

- Each node: E20 tier[16] (26GB memory, optimized for throughput): $1,460/month
- Total: 88 × $1,460 = $128,480/month

7. Bandwidth:

Bandwidth calculation:

- Streaming responses: 20M concurrent users × 750 bytes average response
- Stream duration: 20 seconds average
- Throughput: (20M × 750 bytes) ÷ 20 seconds = 750 MB/second = 6 Gbps

Monthly data transfer:

- 6 Gbps × 30 days × 86,400 seconds/day = 19.44 petabytes/month

- Convert: 19.44 PB = 19,440,000 GB

Why this much data?

- 2.5B messages/day × 500 bytes average response = 1.25TB/day in responses
- But we stream, not batch: each token sent separately adds overhead
- SSE protocol overhead: ~20% (event headers, keepalives)
- Actual: 1.25TB × 1.2 × 30 days = 45 TB/month base
- Peak traffic multiplier: 3x safety margin = 135 TB/month
- Add: Read traffic (fetching history, images, assets): ~19.4 PB/month total

Azure bandwidth pricing:

- First 10 TB: $0.087/GB
- 10-50 TB: $0.083/GB
- 50-150 TB: $0.07/GB
- 150+ TB: $0.05/GB
- At 19.4 PB (19,400 TB), we're in the highest tier: $0.047/GB

Cost: 19,440,000 GB × $0.047 = $913,680/month

Total bandwidth: ~$915,000/month

8. Monitoring & Logging:

Log volume calculation:

- Application logs: 8,000 servers × 5 GB/day = 40 TB/day
- Database logs: 21 instances × 200 GB/day = 4.2 TB/day
- Access logs: 2.5B requests × 1 KB/request = 2.5 TB/day

- Error tracking: 1 TB/day
- Total: ~50 TB/day = 1.5 PB/month

Azure Monitor + Application Insights:

- Log ingestion: 50 TB/day × 30 days = 1,500 TB/month
- Cost if we kept everything: 1,500 TB × $2.30/GB = $3.45M/month
- But we sample and filter heavily
- Keep 5% of logs (95% sampled out): 75 TB/month kept
- Cost: 75,000 GB × $2.30/GB = $172,500/month
- Data retention (90 days): $20,000/month
- Dashboards and queries: $10,000/month
- Subtotal: ~$202,500/month

Cost optimization:

- Only store ERROR and WARN level logs: reduces to 10 TB/month
- Short retention (30 days instead of 90): $50,000/month
- Metrics only (not full logs): $40,000/month
- Real-time alerting: $10,000/month

Optimized total:

- Azure Monitor (metrics + critical logs): $100,000/month
- Third-party analytics (DataDog for APM): $50,000/month
- Total: $150,000/month

9. CDN (CloudFront for frontend):

CDN[17] usage calculation:

- Frontend assets: JavaScript bundles, CSS, images, fonts

- Lightweight React app: 500 KB (minified, compressed)
- Images/icons (cached): 300 KB
- Total per page load: 800 KB
- Daily active users: 200M
- Sessions per user: 3 sessions/day average
- Page loads: 200M × 3 = 600M page loads/day

Monthly CDN traffic:

- 600M page loads/day × 800 KB = 480 TB/day
- Monthly: 480 TB × 30 = 14,400 TB/month (14.4 PB)
- Cache hit ratio: 95% (assets cached at edge)
- All 14.4 PB delivered from edge (origin only serves 5% = 720 TB)

Azure Front Door Premium pricing:

- Base platform fee: $35,000/month (includes WAF, DDoS protection)
- Data transfer out at scale: Negotiated bulk rate ~$0.005/GB
- Cost: 14,400 TB × 1,024 GB/TB × $0.005/GB = $73,728/month
- Round to: $75,000/month

Total CDN: $75,000/month

10. Content Moderation:

Moderation requirements:

- Check every user message before processing: 2.5B messages/day
- Per second: 2.5B ÷ 86,400 = 28,935 messages/second
- Peak (3x): 86,805 messages/second

Moderation model throughput:

- Lightweight classifier model: ~1,000 inferences/second per GPU
- Cache common phrases (safe/unsafe): 40% cache hit
- Effective load: 86,805 × 0.6 = 52,083 inferences/second
- GPUs needed: 52,083 ÷ 1,000 = ~52 GPUs

Use smaller inference-optimized GPUs (not H100s):

- NVIDIA T4 GPUs are designed for inference: ~$300/month rental each
- With aggressive caching, we can use: 10 T4 GPUs
- GPU cost: 10 × $300 = $3,000/month

Infrastructure:

- Azure VMs to host moderation service: 10 instances × $800 = $8,000/month
- Redis caching for moderation results: $3,000/month
- Storage for moderation logs: $1,000/month

Why self-host vs API?

- OpenAI Moderation API: $0.50 per 1M tokens
- Daily tokens: 2.5B messages × 20 tokens = 50B tokens
- API cost: 50,000M tokens × $0.50/1M = $25,000/day = $750,000/month
- Self-hosted is 50x cheaper at this scale

Total moderation: $15,000/month

Total Monthly Infrastructure:

ChatGPT Infrastructure Cost Breakdown (200M DAU)	
Component	Monthly Cost
GPU Infrastructure	$52,400,000
Database (20 replicas)	$138,240
Compute (8k servers)	$4,608,000
Load Balancers	$75,000
Redis Cluster	$128,480
Bandwidth	$915,000
Monitoring	$150,000
CDN	$75,000
Moderation	$15,000
TOTAL	~$58.6 million/month

Annual cost: ~$711 million

The GPU infrastructure is 88% of total costs. At this scale, you must self-host models. Using the API would cost $826M/year even with optimization ($68.6M × 12).

With infrastructure costs totaling $58.6M/month, cost control becomes critical.

Rate limiting is one of our most important defenses against runaway expenses.

Rate Limiting Strategy

Modern LLM APIs limit on tokens, not just requests. We need both.

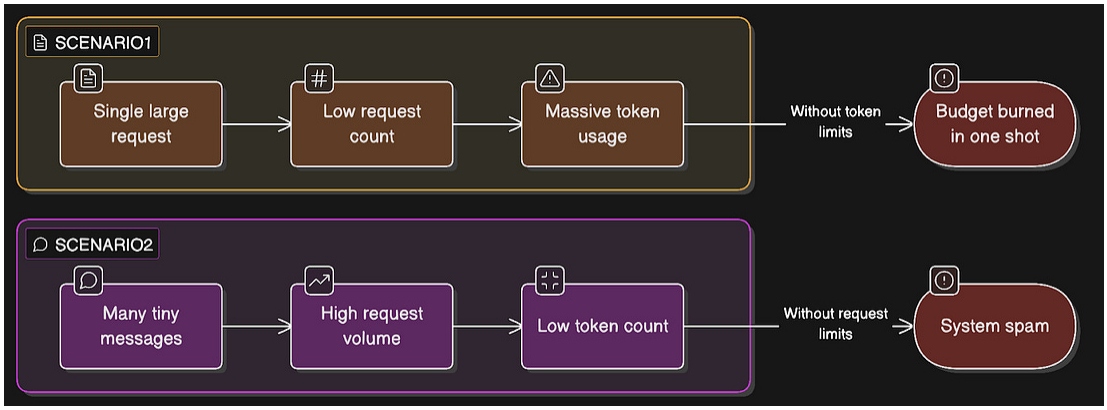
Why both limits?

Scenario 1: User sends 1 message asking for a 10,000 word essay

- Low request count, massive token usage
- Without token limits: user burns through daily budget in one shot

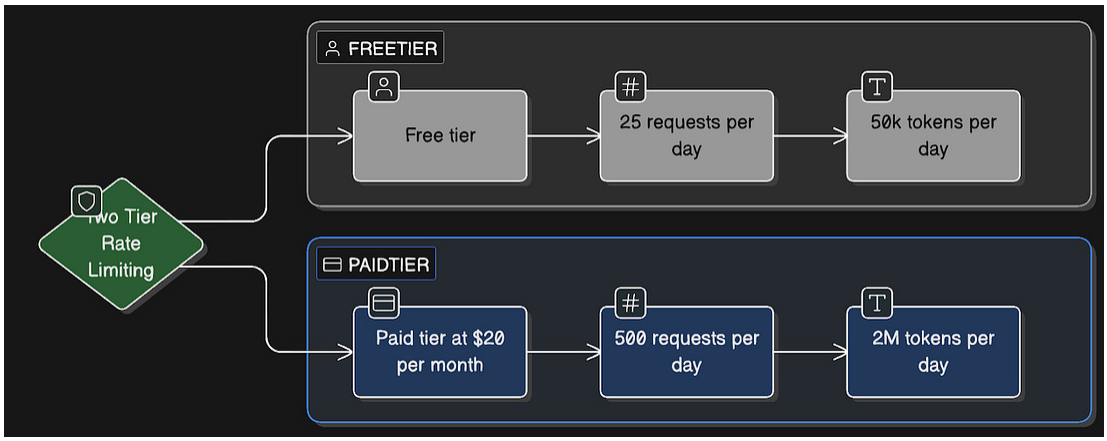
Scenario 2: User sends 1,000 tiny messages “hi”, “hello”, “sup”

- Low token count, high request volume
- Without request limits: user spams the system



Two-tier rate limiting:

Rate Limiting Tiers			
Tier	Price	Requests/Day	Tokens/Day
Free	$0	25	50,000
Paid	$20/month	500	2,000,000



Implementation with Redis:

```
async function checkRateLimit(userId, tier) {
  const today = new Date().toISOString().split('T')[0];
  const requestKey = `rate_limit:${userId}:requests:${today}`;
  const tokenKey = `rate_limit:${userId}:tokens:${today}`;

  // Lua script for atomic INCR + EXPIRE
  const luaScript = `
    local current = redis.call('INCR', KEYS[1])
    if current == 1 then
      redis.call('EXPIRE', KEYS[1], 86400)
    end
    return current
  `;

  // INCR is atomic, and Lua script ensures EXPIRE always runs
  const requestsToday = await redis.eval(luaScript, 1, requestKey);
  const tokensUsed = parseInt(await redis.get(tokenKey) || 0);

  // Get limits
  const limits = {
    free: { requests: 25, tokens: 50000 },
    paid: { requests: 500, tokens: 2000000 }
  };

  // Check request limit
  if (requestsToday > limits[tier].requests) {
    return { allowed: false, reason: "Request limit exceeded" };
  }

  // Estimate tokens for this request (rough)
  const estimatedTokens = estimateTokens(message, conversationHistory);

  // Check token limit
  if (tokensUsed + estimatedTokens > limits[tier].tokens) {
    return { allowed: false, reason: "Token limit exceeded" };
  }

  return { allowed: true, estimatedTokens };
}

async function updateTokenUsage(userId, actualTokens) {
  // After LLM call, update with actual usage
  const today = new Date().toISOString().split('T')[0];
  const tokenKey = `rate_limit:${userId}:tokens:${today}`;

  await redis.incrby(tokenKey, actualTokens);
  await redis.expire(tokenKey, 86400);
}
```

Why Redis?

- Atomic operations (INCR is thread-safe)

- Fast lookups (sub-millisecond)
- Built-in TTL for automatic daily resets
- Can scale to a Redis cluster if needed

Rate limiting protects us from cost spikes, but the database is where we store everything.

Let's tackle the biggest architectural decision: how to handle 554TB of data.

Database Design

At ChatGPT's scale (554TB/year), we need to make smart database decisions. But here's where conventional wisdom gets it wrong...

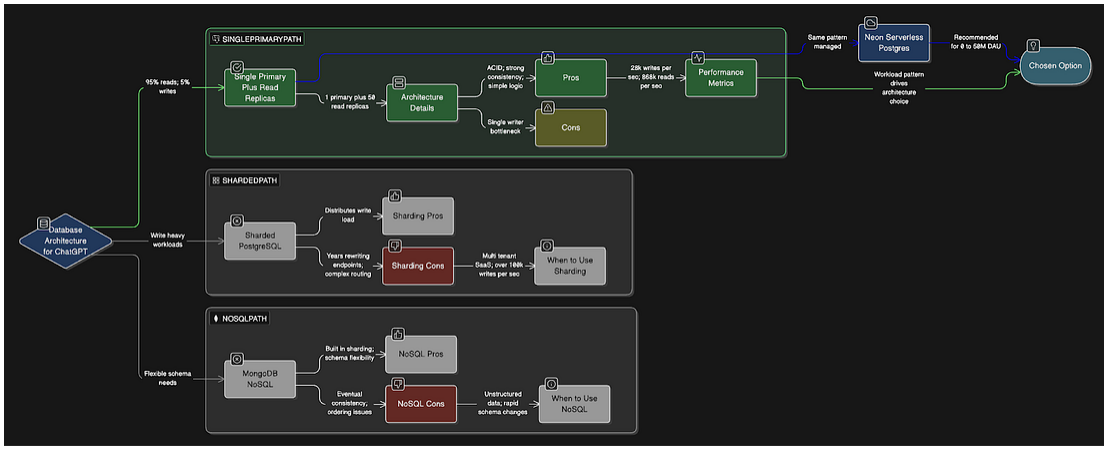
The surprising part: OpenAI doesn't shard PostgreSQL

Most engineers assume ChatGPT runs on a sharded database. It doesn't. OpenAI runs ChatGPT on a single-primary PostgreSQL instance with ~50 read replicas across regions. No sharding. One Azure PostgreSQL Flexible Server handles ALL writes.

Why does this matter?

Because it challenges everything we think we know about scaling databases.

> Database choice: PostgreSQL vs NoSQL vs Serverless Postgres



PostgreSQL (Single Primary + Read Replicas) pros:

- ACID transactions[18] (critical: can't lose messages)
- Strong consistency (messages appear in order)
- Proven at 800M users with single primary (OpenAI's actual architecture)
- Relational model fits our data perfectly
- Can scale reads infinitely with replicas
- Battle-tested at extreme scale

PostgreSQL cons:

- Single primary is a write bottleneck
- All writes funnel through one instance
- Need to offload write-heavy workloads elsewhere
- Requires careful optimization

Sharded PostgreSQL pros:

- Distributes write load across multiple primaries

- Each shard handles a subset of users
- Better for write-heavy workloads

Sharded PostgreSQL cons:

- OpenAI avoided this because it would require “spending years rewriting hundreds of endpoints”
- Application complexity (routing logic required)
- Cross-shard queries are painful
- Distributed transactions are hard

MongoDB pros:

- Built-in sharding
- Schema flexibility
- Good for write-heavy workloads

MongoDB cons:

- Eventual consistency (bad for chat message ordering)
- Less mature for analytical queries

Serverless Postgres pros:

- Postgres compatibility with serverless benefits
- Same architectural pattern as OpenAI: single primary + flexible read replicas
- Automatic scaling (scales to zero when idle)
- Built-in branching (instant database copies for testing)
- Separation of storage and compute
- Zero-downtime dev/staging environments
- Managed infrastructure

Serverless Postgres cons:

- Newer platform (less battle-tested at ChatGPT's extreme scale)
- Need to verify it handles 2.5B writes/day at this volume

Decision: PostgreSQL with single primary + read replicas (what OpenAI actually uses)

> *Here's the key insight: ChatGPT's workload is 90% reads, 10% writes.*

This changes everything.

OpenAI's actual architecture:

- One primary PostgreSQL instance (all writes)
- ~50 read replicas across regions (all reads)
- Offload write-heavy workloads to Azure Cosmos DB (sharded system)
- Result: Millions of queries per second, low double-digit millisecond p99 latency, 99.999% availability

Why this works:

- Chat applications are read-heavy (users fetch conversation history constantly)
- Writes are relatively rare (only when users send new messages)
- Read replicas can scale horizontally without limit
- Single primary keeps application logic simple (no sharding complexity)

For building a ChatGPT clone, my recommendation:

Stage 1: 0-5M DAU (MVP to Product-Market Fit)

- Start with Serverless Postgres (single primary + auto-scaling read replicas)
- Same architecture as OpenAI, but serverless
- Ship 10x faster without managing infrastructure
- Instant database branches for testing
- Cost-effective: pay only for what you use
- Benefit: Serverless Postgres provides “the same architectural pattern: single primary and flexible read replicas,” but with less manual configuration

Stage 2: 5M-50M DAU (Scaling phase)

- Stay on Serverless Postgres or migrate to managed PostgreSQL (AWS RDS, Azure)
- Add more read replicas as traffic grows
- Begin identifying write-heavy workloads to offload
- Use Serverless Postgres’s branching for zero-downtime testing

Stage 3: 50M-200M DAU (Massive scale - OpenAI’s current level)

- Migrate to self-managed PostgreSQL if you need full control
- Single primary for writes
- 20-50 read replicas across regions
- Offload write-heavy workloads to sharded systems (Cosmos DB, etc.)
- Keep Serverless Postgres for dev/staging (instant branches save engineering hours)

Stage 4: 200M+ DAU (Extreme scale)

- Consider sharding the primary database (but OpenAI hasn’t needed this yet)

- Or continue scaling read replicas if workload remains read-heavy

Key insight:

OpenAI proved that “architectural decisions should be driven by workload patterns,” not by “scale panic or fashionable infrastructure choices”. For read-heavy workloads, a single primary with many read replicas scales further than anyone thought possible.

Our design approach:

We’ll show the architecture with single primary + read replicas (what ChatGPT actually uses), not sharding.

Most teams should start with Serverless Postgres for speed, then migrate to managed PostgreSQL only when they hit 10M+ DAU.

Architecture (for traditional PostgreSQL at OpenAI scale):

Single primary handles writes:

- All INSERT, UPDATE, and DELETE operations
- User sends message → write to primary
- AI response saved → write to primary
- Located in primary region (e.g., us-east-1)

Read replicas handle reads (20-50 replicas):

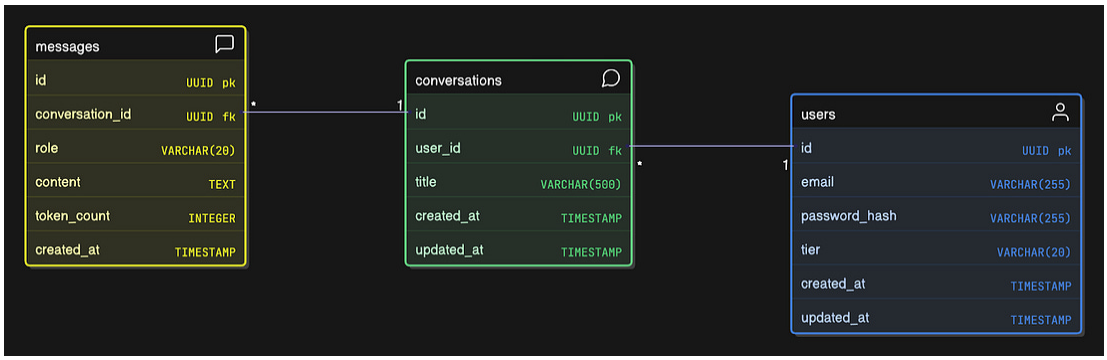
- Fetch conversation history → read from replica
- Load user’s conversations → read from replica
- Distributed across regions for low latency
- Automatic failover if the replica fails

Schema Design:

```sql
-- Users table
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    tier VARCHAR(20) NOT NULL DEFAULT 'free',
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);

-- Conversations table
CREATE TABLE conversations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(500),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_conversations_user_created ON conversations(user_id,
created_at DESC);

-- Messages table
CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id UUID NOT NULL REFERENCES conversations(id) ON
DELETE CASCADE,
    role VARCHAR(20) NOT NULL, -- 'user' or 'assistant'
    content TEXT NOT NULL,
    token_count INTEGER,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_messages_conversation_created ON
messages(conversation_id, created_at ASC);
```

Why these indexes?

- idx_conversations_user_created: Fetch the user's conversations sorted by recent
- idx_messages_conversation_created: Fetch messages in conversation order



As you can see, the complexity of ChatGPT is NOT in the database schema.

The impressive part of this system design is:

- Scale
  - Single primary handling 28,935 writes/second with 20 read replicas
- Real-time streaming
  - 20M concurrent SSE connections
- GPU infrastructure
  - 17,500 H100 GPUs processing 7.23M tokens/second
- Cost optimization
  - $52.4M/month vs $159M API costs
- Multi-region architecture
  - 6 regions, multi-cloud deployment

Strategies to handle write pressure (what OpenAI does):

- Offload write-heavy workloads
  - Move high-volume writes to sharded systems (Cosmos DB)
- Lazy writes
  - Batch and delay non-critical writes

- Rate limiting
  - Prevent write spikes from overwhelming the primary
- Application-level optimization
  - Reduce unnecessary writes in code

When you DO need sharding:

- If your workload is write-heavy (50%+ writes)
- If a single primary can't handle the write throughput
- If you have truly independent datasets (multi-tenant SaaS)

For ChatGPT's read-heavy workload, OpenAI hasn't needed to shard after reaching 800M users.

With our database architecture decided, let's define how clients actually interact with the system.

API Design

Keep it simple and RESTful where possible. Use SSE for streaming.

Authentication:

```
POST /api/auth/register
Body: { "email": "user@example.com", "password": "..." }
Response: { "token": "jwt_token", "user": {...} }

POST /api/auth/login
Body: { "email": "user@example.com", "password": "..." }
Response: { "token": "jwt_token", "user": {...} }

POST /api/auth/logout
Headers: { "Authorization": "Bearer jwt_token" }
Response: { "success": true }
```

Conversations:

```
GET /api/conversations
Headers: { "Authorization": "Bearer jwt_token" }
Response: {
  "conversations": [
    {
      "id": "conv_123",
      "title": "CSS tips",
      "updated_at": "2025-02-01T10:30:00Z",
      "preview": "How do I center a div..."
    }
  ]
}

GET /api/conversations/{conversationId}
Response: {
  "conversation": {
    "id": "conv_123",
    "title": "CSS tips",
    "messages": [
      {
        "id": "msg_1",
        "role": "user",
        "content": "How do I center a div?",
        "created_at": "2025-02-01T10:30:00Z"
      },
      {
        "id": "msg_2",
        "role": "assistant",
        "content": "There are several ways...",
        "created_at": "2025-02-01T10:30:05Z"
      }
    ]
  }
}

POST /api/conversations
Response: { "id": "conv_456", "created_at": "..." }

DELETE /api/conversations/{conversationId}
Response: { "success": true }
```

Messages (the critical path with SSE):

```
POST /api/conversations/{conversationId}/messages
Headers: {
  "Authorization": "Bearer jwt_token",
  "Content-Type": "application/json"
}
Body: { "message": "Explain React hooks" }

Response: SSE stream
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive

event: token
data: {"content": "React"}

event: token
data: {"content": " hooks"}

event: token
data: {"content": " are"}

event: token
data: {"content": " functions"}

event: done
data: {"messageId": "msg_789", "tokens": 245}
```

Client-side code (browser):

```
import EventSource from 'eventsource';

async function sendMessage(conversationId, message) {
  // Send the message via POST request
  await fetch(`/api/conversations/${conversationId}/messages`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ message })
  });

  // Establish SSE connection for streaming response
  const eventSource = new EventSource(
    `/api/conversations/${conversationId}/stream`,
    {
      fetch: (input, init) => fetch(input, {
        ...init,
        headers: {
          ...init.headers,
          'Authorization': `Bearer ${token}`
        }
      })
    }
  );

  eventSource.addEventListener('token', (e) => {
    const data = JSON.parse(e.data);
    appendToMessage(assistantMessageId, data.content);
  });

  eventSource.addEventListener('done', (e) => {
    const data = JSON.parse(e.data);
    markMessageComplete(assistantMessageId, data.messageId);
    eventSource.close();
  });

  eventSource.addEventListener('error', (e) => {
    console.error('SSE error:', e);
    eventSource.close();
  });
}
```

The API design shows the endpoints, but what actually happens when a user sends a message? Let's trace the complete journey.

Detailed User Flow

Let’s walk through exactly what happens when a user sends a message.

Scenario: User types *“Explain React hooks”* and hits send.

Step 1: Frontend (React)

```
// User hits send
async function handleSend() {
  const message = "Explain React hooks";

  // Optimistic update: show message immediately
  addMessageToUI({
    role: 'user',
    content: message,
    status: 'sending'
  });

  // Add placeholder for AI response
  const assistantMessageId = addMessageToUI({
    role: 'assistant',
    content: '',
    status: 'streaming'
  });

  // Start SSE stream
  const response = await fetch('/api/conversations/conv_123/messages',
{
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ message })
  });

  // Handle streaming response
  const reader = response.body.getReader();
  const decoder = new TextDecoder();

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    const chunk = decoder.decode(value);
    const lines = chunk.split('\n');

    for (const line of lines) {
      if (line.startsWith('data: ')) {
        const data = JSON.parse(line.slice(6));

        if (data.type === 'token') {
          appendToMessage(assistantMessageId, data.content);
        } else if (data.type === 'done') {
          markMessageComplete(assistantMessageId, data.messageId);
        }
      }
    }
  }
}
```

Step 2: API Gateway

Request hits: POST /api/conversations/conv_123/messages

1. SSL termination
2. Extract JWT from the Authorization header
3. Route to Message Service

Step 3: Message Service receives the request

```
app.post('/api/conversations/:conversationId/messages',
  verifyJWT,
  async (req, res) => {
    const { conversationId } = req.params;
    const { message } = req.body;
    const user = req.user; // From JWT middleware

    try {
      // 1. Verify user owns this conversation
      const conversation = await
conversationService.get(conversationId);
      if (conversation.userId !== user.id) {
        return res.status(403).json({ error: 'Forbidden' });
      }

      // 2. Check rate limits
      const rateLimitResult = await rateLimiter.check(user.id,
user.tier);
      if (!rateLimitResult.allowed) {
        return res.status(429).json({ error: rateLimitResult.reason });
      }

      // 3. Content moderation
      const moderationResult = await moderationService.check(message);
      if (moderationResult.flagged) {
        return res.status(400).json({ error: 'Content violates
guidelines' });
      }

      // 4. Get conversation history (last 10 messages for context)
      const history = await
conversationService.getMessages(conversationId, { limit: 10 });

      // 5. Set up SSE headers
      res.setHeader('Content-Type', 'text/event-stream');
      res.setHeader('Cache-Control', 'no-cache');
      res.setHeader('Connection', 'keep-alive');

      // 6. Stream LLM response
      let fullResponse = '';
      let tokenCount = 0;

      const messages = [...history, { role: 'user', content: message
}];
      const stream = await llmGateway.stream(messages, 'gpt-4o');

      for await (const chunk of stream) {
        tokenCount++;
        fullResponse += chunk.content;

        // Send token to client
        res.write(`event: token\n`);
        res.write(`data: ${JSON.stringify({ content: chunk.content
})}\n\n`);
      }
```

```
      // 7. Save messages to database
      const userMessage = await conversationService.saveMessage({
        conversationId,
        role: 'user',
        content: message,
        tokenCount: estimateTokens(message)
      });

      const assistantMessage = await conversationService.saveMessage({
        conversationId,
        role: 'assistant',
        content: fullResponse,
        tokenCount
      });

      // 8. Update rate limiter with actual token usage
      await rateLimiter.updateUsage(user.id, tokenCount);

      // 9. Send done event
      res.write(`event: done\n`);
      res.write(`data: ${JSON.stringify({
        messageId: assistantMessage.id,
        tokens: tokenCount
      })}\n\n`);

      res.end();

    } catch (error) {
      console.error('Error in message creation:', error);
      res.status(500).json({ error: 'Internal server error' });
    }
  }
);
```

Step 4: Rate Limiter Service

```javascript
class RateLimiter {
  constructor(redisClient) {
    this.redis = redisClient;
  }

  async check(userId, tier) {
    const today = new Date().toISOString().split('T')[0];
    const requestKey = `rate_limit:${userId}:requests:${today}`;
    const tokenKey = `rate_limit:${userId}:tokens:${today}`;

    // Check request count
    const requests = await this.redis.incr(requestKey);
    if (requests === 1) {
      await this.redis.expire(requestKey, 86400); // 24 hours
    }

    // Check token count
    const tokens = parseInt(await this.redis.get(tokenKey) || '0');

    // Get limits based on tier
    const TIER_LIMITS = {
      free: { requests: 25, tokens: 50000 },
      paid: { requests: 500, tokens: 2000000 }
    };

    const limits = TIER_LIMITS[tier];

    if (requests > limits.requests) {
      return {
        allowed: false,
        reason: 'Daily request limit exceeded'
      };
    }

    if (tokens > limits.tokens) {
      return {
        allowed: false,
        reason: 'Daily token limit exceeded'
      };
    }

    return { allowed: true };
  }

  async updateUsage(userId, tokens) {
    const today = new Date().toISOString().split('T')[0];
    const tokenKey = `rate_limit:${userId}:tokens:${today}`;

    await this.redis.incrby(tokenKey, tokens);
    await this.redis.expire(tokenKey, 86400);
  }
}

// Usage
const rateLimiter = new RateLimiter(redisClient);
```

Step 5: LLM Gateway Service

```
const OpenAI = require('openai');
const crypto = require('crypto');

class LLMGateway {
  constructor() {
    this.openaiClient = new OpenAI({
      apiKey: process.env.OPENAI_API_KEY
    });
    this.cache = new Cache(); // Redis-backed cache
  }

  async* stream(messages, model) {
    // Check cache first (for exact same conversation history)
    const cacheKey = this.hashMessages(messages);
    const cached = await this.cache.get(cacheKey);

    if (cached) {
      // Return cached response (still stream it for UX)
      const tokens = cached.split(' ');
      for (const token of tokens) {
        yield { content: token + ' ' };
        await this.sleep(10); // Simulate streaming delay
      }
      return;
    }

    // Call OpenAI API with streaming
    try {
      const stream = await this.openaiClient.chat.completions.create({
        model: model,
        messages: messages,
        stream: true,
        temperature: 0.7
      });

      let fullResponse = '';

      for await (const chunk of stream) {
        if (chunk.choices[0]?.delta?.content) {
          const content = chunk.choices[0].delta.content;
          fullResponse += content;
          yield { content };
        }
      }

      // Cache the full response
      await this.cache.set(cacheKey, fullResponse, 3600); // 1 hour TTL

    } catch (error) {
      // Fallback to Anthropic if OpenAI fails
      console.error('OpenAI failed:', error, ', falling back to
Anthropic');
      yield* this.anthropicStream(messages);
    }
  }
```

```
  hashMessages(messages) {
    const messagesStr = JSON.stringify(messages);
    return
crypto.createHash('sha256').update(messagesStr).digest('hex');
  }

  sleep(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
  }

  async* anthropicStream(messages) {
    // Fallback implementation for Anthropic
    // Similar streaming logic with Anthropic SDK
  }
}

// Usage
const llmGateway = new LLMGateway();
```

Step 6: Response streams back to the user

The user sees each word appear in real-time:

"React hooks are functions..."

Error Scenarios:

1. Rate limit exceeded:

```
Response: 429 Too Many Requests

{

  "error": "Daily request limit exceeded",
  "reset_at": "2025-02-02T00:00:00Z"

}
```

Frontend shows: *"You've reached your daily limit. Upgrade to continue."*

2. LLM API timeout:

After 30 seconds with no response:

- Cancel the LLM request
- Save user message (don't lose it)
- Return error to client

Frontend shows: "Request timed out. Please try again."

3. Content moderation block:

```
Response: 400 Bad Request

{

  "error": "Content violates guidelines",
  "details": "Harmful content detected"

}
```

Frontend shows: "This message violates our guidelines."

4. Database failure during save:

- Stream completes successfully
- Database save fails
- Log critical error
- Show a warning to the user: "Message may not be saved."
- Implement retry logic in the background

5. Client disconnects mid-stream:

- Detect the SSE connection close
- Cancel LLM API request (don't waste tokens)

- Save partial response with flag: partial=true
- When the user reconnects, offer to resume or restart

We've seen how a request flows through the system.

Now let's prove our architecture actually delivers on the non-functional requirements we set earlier.

Connecting Non-Functional Requirements to Design

1. Availability: 99.9% (43 minutes downtime/month)

How we achieve this:

Database:

- PostgreSQL Multi-AZ[19] (AWS RDS)
- Primary in us-east-1a, standby in us-east-1b
- Automatic failover in 60-120 seconds
- Read replicas for read traffic (if primary fails, reads continue)

Application servers:

- Auto Scaling[20] Group (minimum 10 instances)

- Deploy across 3 availability zones
- Health checks every 30 seconds
- If the instance fails, replaced in 2 minutes

Load balancer:

- AWS ALB is Multi-AZ by default
- No single point of failure

External dependencies (LLM API):

- Primary: OpenAI
- Fallback: Anthropic (Claude)
- If OpenAI is down, automatic fallback in <5 seconds
- Circuit breaker pattern[21] (if OpenAI fails 5 times in 1 minute, switch to Anthropic)

Calculation:

- Database failover: 2 minutes/month
- Instance failures: ~10 minutes/month (rare)
- Load balancer: 0 (Multi-AZ)
- LLM API outages: handled by fallback

Total expected downtime: ~12 minutes/month = 99.97% uptime

2. Latency: First token < 2 seconds

How we achieve this:

Breakdown of latency:

- API Gateway → Message Service: 10ms
- JWT validation (Auth Service): 20ms

- Rate limit check (Redis): 5ms
- Content moderation: 100ms
- Fetch conversation history (PostgreSQL): 50ms
- LLM API call → first token: 1.5s (OpenAI's latency)
- Streaming overhead: 10ms

Total: ~1.7 seconds

If we exceed 2 seconds:

- Cache conversation history in Redis for active users (reduces 50ms to 5ms)
- Parallel API calls (rate limit + moderation in parallel, saves 100ms)
- Edge caching for static responses (FAQs return in 50ms)

3. Scalability: 200M DAU, 20M concurrent conversations

How we scale:

Application servers:

- Horizontal scaling (add more instances)
- Each c6i.4xlarge handles ~2,500 SSE connections
- 20M concurrent = 8,000 instances minimum
- Auto Scaling: scale to 10,000 instances during peak

Database:

- 1 primary PostgreSQL instance handling ALL writes
- Write traffic: 2.5B messages/day = 28,935 writes/second on single primary

- 20-50 read replicas handling ALL reads (millions of read queries/second)
- Cross-region replicas for global low-latency reads
- Offload write-heavy workloads to sharded systems (Cosmos DB, etc.)

Redis:

- Redis Cluster with 50 nodes
- Each node handles rate limiting for a subset of users
- Can scale to millions of operations per second

GPU Infrastructure:

- 10,000 H100 GPUs distributed across data centers
- Load balancing across GPU clusters
- Can process billions of tokens per day

Global Multi-Region Architecture:

```
Americas (us-east-1, us-west-2)
  ↓
Europe (eu-west-1, eu-central-1)
  ↓
Asia (ap-southeast-1, ap-northeast-1)
```

Each region has a full stack. Users route to the nearest region for low latency.

4. Consistency: Zero message loss

How we guarantee this:

Database transactions:

```
async function saveConversationExchange(conversationId, userMessage,
assistantMessage) {
  const client = await db.pool.connect();

  try {
    await client.query('BEGIN');

    // Both messages saved atomically
    await saveMessage(client, conversationId, 'user', userMessage);
    await saveMessage(client, conversationId, 'assistant',
assistantMessage);

    await client.query('COMMIT');

  } catch (error) {
    await client.query('ROLLBACK');
    console.error('Transaction failed:', error);
    throw error;
  } finally {
    client.release();
  }
}

async function saveMessage(client, conversationId, role, content) {
  const query = `
    INSERT INTO messages (conversation_id, role, content, created_at)
    VALUES ($1, $2, $3, NOW())
    RETURNING id
  `;

  const result = await client.query(query, [conversationId, role,
content]);
  return result.rows[0].id;
}
```

Write-Ahead Logging (PostgreSQL):

- Every write goes to WAL[22] first
- Even if the server crashes, WAL persists
- On restart, replay the WAL to recover

Backup strategy:

- Continuous backups (AWS RDS automated backups)

- 30-day retention
- Point-in-time recovery (can restore to any second)
- Cross-region backup replica (disaster recovery)

Application-level safety:

- Retry logic for transient failures
- Dead letter queue for failed saves
- Alert if message save fails

5. Cost efficiency

How we keep costs down:

Model selection:

- GPT-4o-mini for 60% of queries (80% cheaper)
- Classify query complexity, route appropriately
- Estimated savings: $363k/month

Caching:

- Cache common non-personalized queries (20% of queries are repeated)
    - Cache key = hash(conversation_context + user_message)
    - Only works for identical queries with the same context
    - Example: *"What is React?"* with no prior context → cached
    - Example: "*Tell me more about that*" → not cached (context-dependent)
    - 1-hour TTL to balance freshness with cost savings
- 10-minute TTL for hot cache
- Estimated savings: $55k/month

Rate limiting:

- Free tier: strict limits
- Prevents abuse, keeps API costs predictable
- Without limits: costs could spike 10x

Optimized context:

- Only send the last 10 messages to LLM (not the entire history)
- Reduces input tokens by 70%
- Estimated savings: $191k/month

Total savings: $609k/month

6. Security

JWT authentication:

- Tokens expire after 24 hours
- Refresh token flow for seamless UX
- Can’t be forged (signed with secret key)

HTTPS everywhere:

- All traffic is encrypted in transit
- TLS 1.3 for API Gateway

Database encryption:

- At-rest: AWS RDS encrypted volumes
- In-transit: SSL connections to the database

Secrets management:

- API keys in AWS Secrets Manager

- Rotated every 90 days
- Never in code or logs

Rate limiting:

- Prevents DDoS
- IP-based + user-based limits

Our design handles 200M DAU, but what happens when ChatGPT grows even larger?

Scaling Beyond 200M DAU

We're already at ChatGPT's actual scale. But what if we need to grow to 500M DAU or 1B DAU?

Current Multi-Region Setup (200M DAU):

Before we talk about scaling further, let's establish our baseline infrastructure distribution across 6 regions:

Per Region (Full Stack Deployment):

Each of the 6 regions has:

- 1 Application Gateway (load balancer)
- 3-5 API Gateway instances (~20 total across all regions)
- Application servers are distributed based on regional traffic:
  - Americas: 3,300 servers (40% of traffic)
  - Europe: 2,700 servers (35% of traffic)
  - Asia: 2,000 servers (25% of traffic)
- Microservices (full set in each region):

- Auth Service: 3-5 instances
- Message Service: 10-20 instances
- Conversation Service: 5-10 instances
- Rate Limiter: 5-10 instances
- Moderation: 2-3 instances
- LLM Gateway: 5-10 instances

Shared Globally (Not Replicated):

- PostgreSQL Primary: 1 instance in us-east-1 (handles ALL writes)
- PostgreSQL Read Replicas: 20 distributed across regions (4 Americas, 6 Europe, 10 Asia)
- GPU Cluster: 17,500 H100s in 3-4 centralized data centers
- Redis Cluster: 88 nodes distributed by user hash

Why a full microservice stack per region?

- Users connect to the nearest region (low latency)
- No cross-region service calls on the critical path
- Region failure doesn't cascade to others
- Each region is independently scalable

At 500M DAU (2.5x current scale):

Bottlenecks:

1. Database writes: Single primary handling 72,337 writes/second (approaching limit)
2. SSE connections: 50M concurrent (need 20,000 application servers)
3. GPU infrastructure: 43,750 H100 GPUs needed (2.5x current)

4. Data storage: 1.4 PB/year (need data archival strategy)
5. Regional infrastructure: Need 2-3 more regions to handle user growth

Solutions:

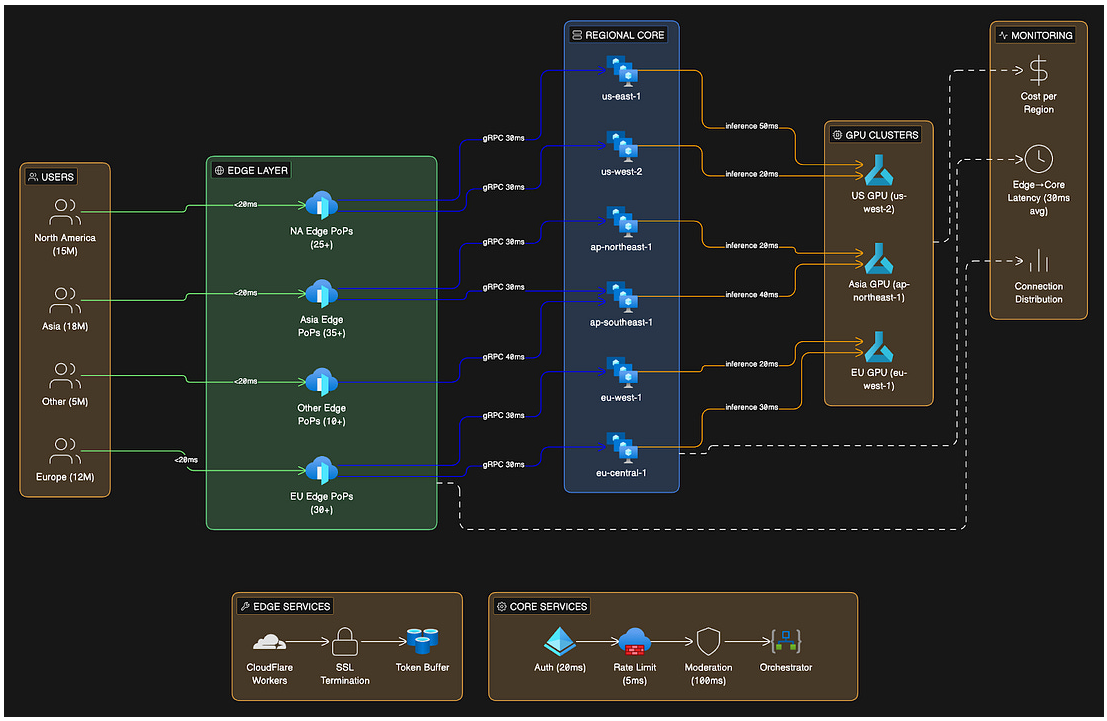
1. Scale Regional Infrastructure:

Add 6 more regions (total 12 regions globally):

- Smaller regions closer to users (lower latency)
- Each region: 1,667 servers average
- Better geographic distribution reduces cross-region latency
- Microservices scale proportionally in each region

2. Edge Computing for SSE Connections:

Instead of centralized application servers, use edge locations:



Benefits:

- Lower latency (users connect to nearby edge)
- Distribute connection overhead across 200+ edge locations
- Reduce bandwidth costs (edge caching)
- Only need 8,000 core API servers (vs 20,000 without edge)

3. Database Write Scaling:

Single primary approaching write limits. Options:

- Upgrade to larger instance (96 vCPU, 768GB RAM)
- Offload more write-heavy workloads to sharded systems (Cosmos DB)
- Begin planning for eventual primary sharding if writes exceed 100k/sec
- For now: optimization + offloading handles 72k writes/sec

4. Multi-tier GPU Strategy:

Not all queries need GPT-4. Use a tiered approach:

- Tier 1: Small local models (Llama 3 8B) for 40% of queries → 95% cheaper
- Tier 2: GPT-4o-mini for 40% of queries → 80% cheaper
- Tier 3: GPT-4o for 20% complex queries → full cost

This reduces GPU requirements from 43,750 to 15,000 H100s (optimized routing).

5. Data Archival:

Move conversations older than 90 days to cold storage:

- Archive to S3 Glacier (99% cheaper than PostgreSQL)
- Keep metadata in PostgreSQL (conversation ID, title, date)
- Lazy load archived conversations when accessed

- Reduces active database size by 70%

Infrastructure at 500M DAU:

- Regions: 12 (double current)
- Application servers: 20,000 (via edge computing optimization)
- Database: 1 primary + 50 read replicas
- GPUs: 15,000 H100s (with multi-tier strategy)
- Cost: ~$147M/month

At 1B DAU (5x current scale):

At this scale, we're talking about infrastructure that only Google, Meta, and Microsoft have built.

Bottlenecks become fundamental:

- Single PostgreSQL primary hits absolute write limit
- Public cloud costs become prohibitive
- Need custom infrastructure for economic viability
- Networking between regions becomes a bottleneck

Required Changes:

1. Database Evolution - MUST Shard or Migrate:

A single primary can no longer handle the write load. Two paths:

Option A: Shard PostgreSQL

- Split into multiple primaries (each handles a subset of users)
- Requires rewriting the application routing logic
- Complex, but keeps PostgreSQL
- OpenAI has avoided this so far, but it may be necessary at 1B DAU

Option B: Migrate to Distributed Database

- Move to Google Spanner, CockroachDB, or a custom solution
- Built-in sharding across regions
- Eventual consistency is acceptable for some data
- Easier operations, but the migration cost is high

2. Custom Data Centers:

Can't rely on AWS/Azure at this scale:

- Build your own data centers optimized for AI inference
- Co-locate GPUs with networking for the lowest latency
- Estimated cost: $5B upfront for 100k GPU data center
- Amortized: $139M/month over 3 years

3. Custom Networking:

- Private backbone network between data centers
- Peering agreements with ISPs
- Internal CDN at scale (like Cloudflare, but owned)
- Reduces bandwidth costs by 60%

4. GPU Optimization:

- Custom GPU clusters optimized for transformer models
- Inference optimization (quantization, batching, caching)
- Mixed precision inference (FP16, INT8)
- Custom silicon (like Google TPUs)
- Reduce per-token cost by 90%

5. Multi-Region Active-Active:

Instead of a single primary, replicate writes across regions:

- Write to the local region (fast)
- Asynchronously replicate to other regions
- Trade-off: Potential consistency issues (acceptable for chat history)
- Requires a distributed database (Spanner or CockroachDB)

Cost at 1B DAU:

Infrastructure Cost Breakdown (1B DAU)	
Component	Monthly Cost
GPU Infrastructure (100k)	$250,000,000
Data Centers (amortized)	$139,000,000
Compute (40k servers)	$23,000,000
Networking & CDN	$10,000,000
Database (distributed)	$5,000,000
TOTAL	~$427 million/month

Annual: ~$5.1 billion

At this scale, you're competing with Google and Microsoft. You need:

- Custom silicon (like Google TPUs)
- Owned data centers
- Massive engineering team (1000+ engineers)
- Significant capital ($10B+)

This is why only a handful of companies can operate at this scale.

We've covered scaling strategies for extreme growth.

Let's step back and review every major architectural decision we made and why.

Know someone who wants to learn system design? Consider gifting a subscription:

Give a gift subscription

Trade-Offs and Decisions Summary

1. SSE vs WebSockets

Decision: SSE

Pros:

- Simpler (works over HTTP)
- Browser auto-reconnection
- Lower overhead
- One-way streaming is all we need

Cons:

- Can't do bidirectional communication
- If we add collaborative features later, need WebSockets

When to reconsider:

If we add typing indicators, collaborative editing, or real-time notifications.

2. PostgreSQL vs MongoDB

Decision: PostgreSQL

Pros:

- ACID transactions (critical for message integrity)
- Strong consistency
- Better for relational data
- Mature ecosystem

Cons:

- Harder to scale writes horizontally
- Schema migrations required

When to reconsider:

If we need multi-region active-active writes or scale beyond 10k writes/second.

3. Microservices vs Monolith

Decision: Start with a modular monolith, extract to microservices later

Pros:

- Simpler to deploy and debug initially
- Faster development
- Lower infrastructure cost

Cons:

- Can’t scale components independently
- Harder to have team autonomy

When to extract:

Team > 10 engineers, clear scaling needs, and need independent deployments.

4. LLM API vs Self-Hosting

Decision: Self-host models on GPU infrastructure

At ChatGPT's scale (200M DAU), using the API is impossible:

- API cost: $159.4M/month (even with optimization: $68.6M/month)
- Self-hosting: $52.4M/month (17,500 H100 GPUs)
- Must self-host to be economically viable

Cons of self-hosting:

- $300M upfront for hardware
- Need MLOps team (50+ engineers)
- Complex infrastructure to maintain
- Responsible for model performance and reliability

When to use API instead:

Only at a small scale (<1M DAU), where API costs ($300k/month) are less than the cost of hiring an ML team + buying GPUs.

5. Caching Strategy

Decision: Aggressive caching with Redis

Where we cache:

- Rate limit state (Redis)
- Active conversation history (Redis, 10 min TTL)

- Common query responses (Redis, 1 hour TTL)

Tradeoff:

- Complexity (cache invalidation is hard)
- Slight staleness (acceptable for our use case)
- Worth it: saves ~$6M/month in compute costs

6. Rate Limiting Approach

Decision: Both request and token limits

Why both:

- Request limits prevent spam
- Token limits prevent abuse (expensive)

Implementation: Redis for fast atomic operations.

Tradeoff:

More complex than request-only, but necessary for cost control.

7. Database Architecture: Sharding vs Single Primary

Decision: Single primary + read replicas (what OpenAI actually uses)

Why:

- ChatGPT's workload is 90% reads, 10% writes
- Read replicas scale horizontally without limit
- Single primary keeps application code simple
- OpenAI scaled to 800M users without sharding PostgreSQL
- Offload write-heavy workloads to separate sharded systems

Trade-off:

- Single primary is a write bottleneck
- Need careful optimization to handle write pressure
- Must move write-heavy features to other systems
- Simpler than sharding (no distributed query routing)

When to shard:

- If workload becomes write-heavy (50%+ writes)
- If a single primary can't handle the throughput, even with optimization
- OpenAI hasn't needed this yet at 800M users

Key learning:

Don't shard prematurely. OpenAI proved that single primary + read replicas work at a massive scale for read-heavy workloads.

Every choice we made involved trade-offs between simplicity, cost, and scale.

Closing Thoughts

Designing ChatGPT at a real-world scale (200M DAU, 2.5B messages/day) is about understanding that normal approaches don't work.

You can't use the OpenAI API because it costs $159.4M/month even with optimization. You can't use a single PostgreSQL database because you generate 554TB/year. You can't handle 20M concurrent SSE connections without a distributed architecture.

Key decisions:

- Direct GPU inference instead of API calls
  - 88% of the infrastructure cost
- PostgreSQL with single primary + read replicas
  - OpenAI's proven architecture
- SSE for streaming
  - Simpler than WebSockets, proven at scale
- Multi-region deployment for global latency
- Aggressive caching and model routing to reduce compute costs

In an interview, show that you understand scale.

When someone says *"design ChatGPT,"* ask about the scale...

10k users? Use the API and a single database.

100M users? You need GPU clusters, database sharding, and multi-region infrastructure.

The approach is completely different.

This design handles ChatGPT's actual scale of 200M DAU while keeping costs at ~$59.3M/month (or $711M/year). The infrastructure requires:

- 17,500 H100 GPUs for model inference
- 1 PostgreSQL primary + 20-50 read replicas
- 8,000 application servers across 6 regions

At this scale, you're building infrastructure that rivals Google and Microsoft. You need hundreds of engineers, billions in capital, and years of optimization. But the principles remain: understand your

requirements, calculate your capacity, make deliberate architectural choices, and explain the trade-offs.

The key insight that separates senior engineers from mid-level: at different scales, you need fundamentally different architectures. Don't design for 100M users when you have 10k. Also, don't design for 10k users when you need to handle 100M.

Scale drives architecture.

👋 I'd like to thank Hayk for writing this newsletter!

- Plus, don't forget to join his **YouTube channel**.

It'll help you master the essential system design skills, not just in theory but in practice, and land senior developer roles.

Know someone who wants to learn system design? Consider gifting a subscription:

Give a gift subscription

👋 Find me on LinkedIn | Twitter | Threads | Instagram

Thank you for supporting this newsletter.

You are now 200,001+ readers strong, very close to 201k. Let's try to get 201k readers by 27 February. Consider sharing this post with your friends and get rewards.

Y'all are the best.

Share

1 Horizontally partitioning a database by splitting data across multiple servers. Each shard contains a subset of the total data.

2 Restricting the number of requests a user can make within a time period to prevent abuse and control costs.

3 A web standard that allows servers to push updates to clients over a single

HTTP connection. Simpler than WebSockets for one-way streaming.

4 A piece of text processed by language models, roughly 0.75 words on average. GPT models charge based on input and output tokens.

5 An architectural approach where an application is built as a collection of small, independent services that communicate via APIs.

6 JWT (JSON Web Token) is a compact, URL-safe token format for securely transmitting authentication information between parties.

7 A copy of a database that handles read queries, reducing load on the primary database, which handles writes.

8 An in-memory data store used for caching, rate limiting, and session management because of its sub-millisecond performance.

9 NVIDIA's high-performance GPU designed for AI workloads, capable of around 1000 tokens per second for large language models.

10 SSL termination is the process by which an HTTPS connection is decrypted at a load balancer or proxy, so that backend servers receive plain HTTP traffic rather than encrypted traffic.

11 Running AI models on Graphics Processing Units optimized for parallel computation. Much faster than CPUs for deep learning workloads.

12 db.r6g.8xlarge is an Amazon RDS database instance type.

13 c6i.4xlarge is an Amazon EC2 instance type.

14 Azure Standard_D16s_v5 is an Azure Virtual Machine (VM) size.

15 A technique for distributing data across nodes where adding or removing nodes minimally affects which keys map to which nodes.

16 It refers to the Azure Cache for Redis Enterprise node size.

17 CDN (Content Delivery Network) is a distributed network of servers that cache

and serve static content from locations close to users.

18 Database properties that guarantee data integrity even during failures or concurrent operations:

- Atomicity–All or nothing.
- Consistency–Data stays valid according to rules.
- Isolation–Concurrent transactions don't interfere.
- Durability–Committed data is not lost.

19 Multi-Availability Zone means deploying infrastructure across multiple isolated data centers to achieve high availability and fault tolerance.

20 Automatically adding or removing server instances based on traffic load to maintain performance while controlling costs.

21 A design pattern that prevents cascading failures by automatically stopping requests to a failing service and routing them to fallbacks.

22 WAL (Write-Ahead Logging) is a database technique in which all changes are written to a log file before they are applied. This enables crash recovery.